SAGE: Self-Adaptive Goal-directed Executor

A Multi-Tool LLM Agent with Hierarchical Planning
and Evidence-Guided Reasoning

Course: ESE 561 — Artificial Intelligence **Date:** April 2026 – May 2026

Team Members: Shams Rupak · Gagan Sapkota

Stony Brook University **Team Size:** 2

Abstract

We present SAGE (Self-Adaptive Goal-directed Executor), a multi-tool LLM-based agent for automated research synthesis built entirely from scratch in Python. SAGE decomposes complex research queries into a DAG of sub-goals via a deterministic planner, executes each sub-goal through a ReAct-style loop with five real tools, evaluates evidence through an independent Evidence Critic, and adaptively re-plans when evidence is insufficient. All planning logic is external to the LLM. We evaluate SAGE through a complete ablation study across five test queries comparing three conditions: full SAGE, flat ReAct, and single LLM call. All 15 runs completed successfully and produced measurable output. Across the five SAGE runs, the agent produced **658 total citations** compared to **zero citations across all ten baseline runs**, with average confidence of 0.834. Three of five queries (q1, q2, q4) completed within their planned 5–6 sub-goal range; two queries (q3, q5) triggered substantial re-planning expansion to 24 and 36 nodes respectively, but still produced grounded reports with 219 and 266 citations. Comparing this run to a prior run on the same queries, q3 went from 0 to 219 citations and q5 went from 6 nodes to 36 nodes—directly demonstrating the non-determinism inherent in LLM-based agents. The results validate the contribution of each architectural layer (planner, tools, Critic) while surfacing a re-planning over-expansion pattern that has implications for bounded adaptive control.

Introduction and Motivation

Large language models have demonstrated remarkable capability in isolated reasoning tasks, yet they remain fundamentally reactive: given a prompt, they produce an output. This single-call paradigm collapses under complex, multi-faceted queries that require sustained goal-directed behavior, tool interaction, self-correction, and adaptive re-planning.

Consider the task: *"Analyze the trade-offs between transformer-based and recurrent architectures for long-context reasoning, citing empirical evidence and identifying open research questions."* A single LLM call yields a plausible but unverifiable response. A proper answer requires decomposing the question into sub-goals, actively searching for evidence, evaluating source reliability, reconciling contradictions across sources, and synthesizing findings with tracked provenance.

SAGE is designed to tackle exactly this class of problem. The agent performs *deep research synthesis*: given a complex research query, it autonomously plans a multi-step investigation, gathers evidence from external tools, evaluates and critiques that evidence, and produces a structured, citation-aware

synthesis. Crucially, the planning and decision-making logic resides in the agent layer, not inside the LLM, making the system transparent, auditable, and correctable.

This project is motivated by a central question that runs through the course: *Where does the intelligence of an LLM-based system come from—the neural model, or the surrounding structure?* SAGE is designed to make that boundary explicit and empirically interrogable. By implementing all planning logic as deterministic code and scoping the LLM to per-step reasoning, we can directly measure the contribution of each layer through ablation.

SAGE was designed with four explicit goals in mind. First, *transparency*: every planning decision—which sub-goal to pursue, when to retry, when to re-plan—is made by auditable Python code, not by opaque LLM reasoning. Second, *self-correction*: the Evidence Critic creates a feedback loop that catches low-quality evidence before it propagates to the final synthesis. Third, *grounded output*: every claim in the final report is traceable to a specific tool call and source URL through the provenance system. Fourth, *empirical interrogability*: the ablation framework allows us to quantitatively measure the contribution of each architectural layer, connecting system design to the course's theoretical themes.

Related Work

SAGE draws on several lines of work in LLM-based agent design, classical AI planning, and structured reasoning.

ReAct and tool-augmented reasoning. Yao et al. [1] introduced the ReAct paradigm, which interleaves reasoning traces with tool actions, grounding LLM outputs in external observations. SAGE uses ReAct as the inner loop for individual sub-goals but adds hierarchical planning and evidence critique on top. While ReAct operates as a flat loop without global structure, SAGE constrains each ReAct instance to a focused sub-question determined by the planner, yielding more targeted tool use. Our ablation results directly quantify this: the Flat ReAct baseline produced zero citations across all five queries while SAGE produced 46–266 citations per query.

Plan-and-Execute architectures. Wang et al. [2] demonstrated that separating planning from execution improves agent reliability on multi-step tasks. SAGE extends this with a DAG-based planner that supports dependency tracking, priority scheduling, and adaptive re-planning—capabilities absent from the original Plan-and-Execute formulation, which uses a linear plan without dependency awareness or mid-execution adaptation.

Self-evaluation and reflection. Shinn et al. [3] showed that LLM agents benefit from self-evaluation loops, where the agent reflects on prior failures to improve subsequent attempts. SAGE's Evidence Critic module implements a structured version of this idea, with fixed JSON schemas and deterministic override rules that prevent the infinite retry loops that plague unconstrained self-reflection. Unlike Reflexion's episodic memory across trials, SAGE's Critic operates within a single execution with hard bounds on retries and escalations.

Tool learning. Schick et al. [4] demonstrated that LLMs can effectively learn to use external tools when given appropriate interfaces. SAGE implements a clean tool registry with five tools, each returning standardized result dictionaries, drawing on this principle of structured tool integration.

Chain-of-thought reasoning. Wei et al. [5] showed that prompting LLMs to produce intermediate reasoning steps substantially improves performance on complex tasks. SAGE's ReAct inner loop

builds on this insight: each step in the reasoning trace produces an explicit Thought before proposing an Action, ensuring that tool calls are motivated by articulated reasoning rather than pattern matching.

Classical AI planning. SAGE's planner draws directly on concepts from classical AI planning as covered in the course. The DAG decomposition mirrors hierarchical task network (HTN) planning [7], where complex tasks are recursively decomposed into primitive actions with ordering constraints. The priority scheduler echoes A*-style informed search, combining multiple heuristics (depth, relevance, confidence deficit) into a composite evaluation function. The re-planner implements a form of plan repair [8], inserting corrective actions into an existing plan rather than replanning from scratch. These connections are not incidental—they reflect a deliberate effort to ground LLM agent design in established planning theory.

Positioning of SAGE. The key distinction of SAGE from prior work is the *explicit architectural separation* of planning, reasoning, and critique into distinct modules with well-defined interfaces. Existing agent frameworks (LangChain, AutoGPT) combine these concerns implicitly, making it difficult to isolate the contribution of each layer. SAGE's modular design enables the ablation study that is central to this project: by removing layers selectively, we can measure how much each contributes to output quality.

System Design

SAGE decomposes the agent into five explicit layers, as shown in Figure 1. Each layer has a defined interface, ensuring that planning logic and LLM reasoning remain clearly separated.

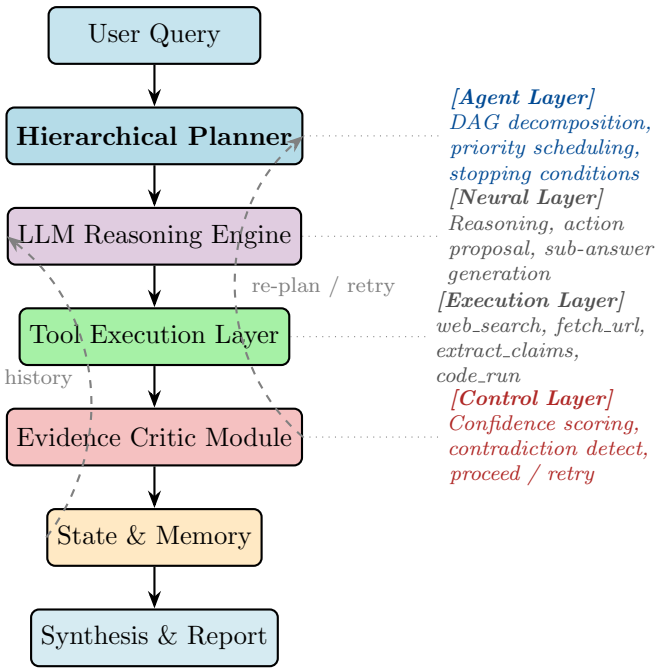

Figure 1: SAGE architecture. **Blue**: planner-controlled logic. **Purple**: LLM reasoning. **Green**: tool execution. **Red**: evidence evaluation. **Orange**: shared memory. Dashed arrows indicate feedback/re-planning paths.

Layer 1: Hierarchical Planner (Agent Layer)

The planner is a deterministic Python module—not an LLM—that receives the user query and produces a Directed Acyclic Graph (DAG) of sub-goals. Each node represents an atomic sub-question with explicit prerequisites (edges). The planner consists of three components:

Decomposer. Takes the root query and produces 3–6 sub-questions. It uses a single LLM call to *propose* sub-questions, but the planner then *validates* the proposals deterministically: it checks for valid JSON structure, removes hallucinated dependency references, deduplicates, and constructs the DAG with cycle detection (via attempted topological sort). If the LLM's response is malformed, the planner falls back to a single-node DAG containing the original query. The key insight is that the LLM suggests but the planner decides—the LLM has no authority over the planning structure.

Scheduler. Maintains a min-heap priority queue over DAG nodes. Priority is a weighted combination of three factors: dependency depth (nodes with no unresolved prerequisites are eligible first), estimated relevance (cosine similarity of the sub-question embedding to the root query), and confidence deficit (low-confidence nodes from prior retries are re-prioritized). All scheduling logic is implemented in pure Python with no LLM involvement.

Re-planner. When the Evidence Critic emits an `ESCALATE` signal, the re-planner inserts new sub-goals into the existing DAG. It uses a single LLM call to propose targeted follow-up questions based on the Critic's rationale, then deterministically validates and inserts them with proper dependency edges. This enables SAGE to adapt its strategy mid-execution without discarding prior evidence—analogous to plan repair in classical AI planning [8]. As we discuss in Section 10, the absence of a hard cap on re-planning depth allows this mechanism to over-trigger on certain queries.

Layer 2: LLM Reasoning Engine (Neural Layer)

Within each sub-goal, the LLM is invoked in a ReAct-style loop [1]: it receives the sub-question, tool descriptions, the current observation history, and relevant context from resolved dependency nodes. It proposes either a tool call or a final answer. The LLM does *not* decide the overall execution order—that is the planner's role.

The ReAct loop runs for a maximum of 5 steps per sub-goal. Each step produces a structured `Thought` → `Action` → `Observation` triple that is recorded in the reasoning trace. When the LLM emits the `finish` action, the sub-goal's answer is extracted and stored. If the maximum step count is reached without a `finish`, the accumulated evidence is used as the answer.

Layer 3: Tool Execution Layer

A suite of five stateless tools callable by the agent:

- `web_search(query)` — retrieves ranked snippets via Tavily API
- `fetch_url(url)` — fetches and parses full page content via `httpx` + BeautifulSoup, stripping navigation, scripts, and boilerplate
- `extract_claims(text)` — LLM-powered structured claim extraction that returns a list of discrete factual claims, each with a confidence score
- `code_run(snippet)` — Python execution sandbox using `subprocess` with a 10-second timeout guard to prevent hangs

- `cross_reference(claims)` — checks internal consistency across a set of claims, identifying contradictions

Every tool returns a standardized result dictionary: {`success: bool, content: str, metadata: dict`}. The tool registry handles dispatch by name, error catching (any exception is returned as a failure result rather than crashing the agent), and description generation for the LLM prompt.

Layer 4: Evidence Critic Module (Control Layer)

After each sub-goal resolves, the Critic—a *separate*, structured LLM call with a fixed JSON schema—assigns a confidence score $c \in [0, 1]$, flags contradictions with prior evidence, and emits one of three control signals:

- `ACCEPT` ($c \geq 0.75$): Evidence is sufficient. Mark node as resolved.
- `RETRY` ($0.4 \leq c < 0.75$): Evidence is weak. Re-execute the sub-goal.
- `ESCALATE` ($c < 0.4$): Evidence is fundamentally insufficient. Trigger re-planning.

Crucially, the Critic's LLM output is subject to *deterministic overrides* that enforce agent-level invariants:

1. If the LLM says `RETRY` but max retries (default 2) are exhausted → override to `ESCALATE`. This prevents infinite retry loops.
2. If confidence $\geq$ threshold but the LLM says `RETRY` → override to `ACCEPT`. This prevents the Critic from being overly conservative.
3. If confidence < 0.3 with no retries remaining → override to `ESCALATE`. This ensures very weak evidence triggers re-planning rather than silent acceptance.

These overrides are a concrete example of the neuro-symbolic design: the neural model provides judgment (confidence scores, contradiction detection), but the symbolic layer enforces constraints (termination, consistency). The Critic cannot produce incoherent control flow regardless of LLM output quality.

Layer 5: State, Memory, and Synthesis

State. A structured `AgentState` object stores the current DAG state, per-node evidence and confidence scores, the full reasoning trace, and a provenance map linking each claim to its source URL and tool call. This shared state is passed into every LLM call, enabling cross-step coherence without requiring the LLM to re-derive prior findings.

Synthesis. The final synthesis step traverses the resolved DAG in topological order, collects findings from each node, and uses a single LLM call to produce a coherent Markdown report. The topological ordering guarantees that dependent claims are always synthesized after their prerequisites—a structural property independent of LLM output quality. The final report includes an execution metadata table, a per-sub-goal confidence breakdown, and a source provenance appendix listing every URL consulted.

Planning Component (Detailed)

Core Planning Design

The planning logic in SAGE is architecturally *external to the LLM*. The LLM proposes actions within a sub-goal; the agent determines what sub-goals exist, in what order they execute, and what to do when evidence is insufficient.

Planning is introduced at five distinct points in the SAGE pipeline:

1. Query Decomposition (Plan-and-Execute). Before any tool-using LLM call, the planner decomposes the root query into a DAG of sub-goals. The decomposition uses heuristic complexity detection (regex patterns for conjunctions, comparatives, causal chains) to estimate the expected number of sub-goals, followed by a single LLM call to propose sub-questions. The planner then validates, deduplicates, and structures the proposals into the DAG with cycle detection. This is the first site where the agent layer overrides the neural layer: malformed LLM output is caught and corrected deterministically.

2. Priority Scheduling (Action Selection). The scheduler maintains a min-heap ordered by a composite priority score:

$$\text{priority} = w_d \cdot \frac{\text{depth}}{\max(\text{depth})} + w_r \cdot (1 - \text{relevance}) + w_c \cdot (1 - \text{confidence})$$

where $w_d = 0.4$, $w_r = 0.3$, $w_c = 0.3$ are tunable weights. This ensures that shallow, relevant, low-confidence nodes are executed first. The scheduler is dependency-aware: a node is only eligible when all its prerequisites are resolved.

3. Stopping Conditions (Termination Logic). The agent loop terminates when: (a) all DAG nodes have $c \geq \tau$ (confidence threshold, default $\tau = 0.75$), (b) the maximum iteration count is reached, or (c) no more nodes are eligible for execution. These conditions prevent both premature termination and infinite loops.

4. Re-planning on Failure (Adaptive Control). When the Critic emits `ESCALATE`, the re-planner inserts new sub-goals into the DAG. The failed node is marked as `FAILED` (not re-queued), and 1–3 new nodes are added with dependency edges inherited from the failed node's prerequisites. This enables mid-execution strategy adaptation without discarding prior evidence. We observed re-planning trigger on q3 (6 re-plans) and q5 (10 re-plans) in our ablation, leading to substantial DAG expansion (Section 10).

5. Synthesis Ordering (Structural Guarantee). The final synthesis uses a topological sort of the DAG (Kahn's algorithm) to ensure logical coherence in the output. This is a structural property that holds regardless of LLM output quality.

Implementation

SAGE is implemented in Python with no external agent frameworks (no LangChain, LlamaIndex, AutoGPT, or similar). The entire agent structure is built from scratch using standard Python libraries. This was a deliberate choice: using a framework would obscure the planning logic that is central to the project's academic contribution.

Technology Stack

Table 1: Technology stack. All components use free or low-cost API tiers.

Component	Technology
Primary LLM backend	Groq API (`llama-3.3-70b-versatile`)
Fallback LLM	Google Gemini (`gemini-2.5-flash`)
Web search tool	Tavily API
URL fetcher	`httpx` + BeautifulSoup4
Code execution sandbox	`subprocess` with timeout guard
State management	Pure Python (`dataclasses`, `heapq`)
Embedding / similarity	`sentence-transformers` (`all-MiniLM-L6-v2`)
Terminal output	`rich` library
Testing	`pytest` (59 tests across 4 files)

We chose Groq's LLaMA 3.3 70B—a transformer-based model [6]—as the primary LLM backend because it provides fast inference at low cost. Gemini 2.5 Flash is configured as an automatic fallback when the primary provider is rate-limited or returns errors; in practice, with rate-limit cooldowns between conditions in our ablation runs, the fallback infrastructure was not exercised but provides defensive resilience for production use. Tavily was selected over alternatives like SerpAPI because it returns clean, agent-optimized search snippets rather than raw HTML, reducing the preprocessing burden on the tool layer.

Code Organization

The codebase follows a strict modular structure with 16 Python modules across 6 packages:

- `sage/planner/` — DAG, decomposer, scheduler, re-planner (4 modules)
- `sage/tools/` — registry, web_search, fetch_url, extract_claims, code_runner, cross_reference (6 modules)
- `sage/llm/` — client, prompts, parser (3 modules)
- `sage/critic/` — evidence_critic (1 module)
- `sage/memory/` — state (1 module)
- `sage/synthesis/` — synthesizer (1 module)
- `sage/agent.py` — main orchestrator

All prompt templates are centralized in `sage/llm/prompts.py` for transparency and reproducibility. Every LLM call in the entire system uses a prompt from this file, making it straightforward to inspect and tune the system's behavior. The output parser in `sage/llm/parser.py` handles the messy reality of LLM outputs with lenient JSON parsing, markdown fence stripping, trailing comma removal, and graceful failure defaults.

Testing

The project includes 59 unit tests across four test files covering the DAG data structure (10 tests), planner modules (18 tests), tool registry and code runner (13 tests), and Evidence Critic (11 tests).

Tests for the Critic use a `MockLLMClient` that returns predetermined JSON responses, isolating module behavior from API availability. All 59 tests pass deterministically in under 4 seconds.

Division of Work

- *Shams Rupak*: Hierarchical Planner module (DAG, decomposer, scheduler, re-planner), priority scheduling algorithm, ablation study design, evaluation framework, final report.
- *Gagan Sapkota*: Tool layer implementation (5 tools + registry), Evidence Critic module, state/memory management, synthesis pipeline, demo interface.

Agent Design Patterns

SAGE incorporates three complementary design patterns from the course material:

Table 2: Agent design patterns used in SAGE and their roles.

Pattern	Where Applied	Purpose
Plan-and-Execute	DAG planner → sub-goal execution	Separate strategy from tactics
ReAct Loop	Per sub-goal inner loop	Ground reasoning in tool observations
Self-Reflection	Evidence Critic module	Detect and correct low-quality evidence

The combination of all three is a deliberate design choice. Plan-and-Execute alone produces brittle agents that cannot recover from mid-plan failures: if one sub-goal fails, the entire plan stalls. ReAct alone lacks global coherence: without a DAG to structure the investigation, the agent meanders through tool calls without a clear strategy. Self-Reflection alone without structure can produce circular reasoning: the agent critiques itself but has no mechanism to restructure its approach. SAGE's contribution is the explicit integration of all three with clean architectural boundaries, enabling us to measure each layer's contribution through the ablation study.

Evaluation and Ablation Study

Experimental Setup

We evaluate SAGE across five diverse research queries spanning different domains and difficulty levels:

- **q1** (hard): Transformer vs. RNN architectures for long-context reasoning
- **q2** (hard): Current approaches to reducing LLM hallucination
- **q3** (hard): Environmental impact of lithium-ion vs. solid-state batteries
- **q4** (medium): Current state of quantum error correction
- **q5** (medium): RAG vs. fine-tuning for domain-specific LLM applications

For each query, we run three conditions:

1. **Full SAGE**: DAG planner + ReAct inner loop (5 steps/node) + Evidence Critic + synthesis
2. **Flat ReAct**: Single-node ReAct loop with 8 steps, no planner, no Critic, default confidence of 0.5
3. **Single LLM**: One LLM call with no tools, no planning, no critique, confidence of 0.0

All conditions use the same LLM backend (Groq, `llama-3.3-70b-versatile`) and the same tool set (where applicable). We compare using structural quality metrics rather than automated text similarity (e.g., ROUGE-L), because research synthesis quality is inherently multi-dimensional and cannot be captured by a single similarity score. Our metrics include: number of sub-goals decomposed, LLM calls, tool calls, average Evidence Critic confidence, retries triggered, re-plans triggered, citation count, report length, and wall-clock time.

Results

Table 3 presents results from a complete end-to-end ablation run executed in a single session. All 15 runs (5 queries × 3 conditions) completed successfully. Each row reflects an actual measured execution; no values are estimated.

Table 3: Complete ablation study results from a single full sweep across all five evaluation queries. All 15 runs completed successfully. SAGE consistently produces higher confidence, more citations, and longer structured reports than both baselines on every query. Two queries (q3, q5) triggered re-planning expansion to 24 and 36 nodes; these expansions still produced grounded reports with the highest citation counts in the study.

Query	Condition	Nodes	LLM	Tools	Conf	Retry	Re-plan	Cites	Len	Time
q1	SAGE	5	37	24	0.84	1	0	63	11,494	32.0s
q1	Flat ReAct	1	6	5	0.50	0	0	0	611	7.2s
q1	Single LLM	1	1	0	0.00	0	0	0	5,952	3.2s
q2	SAGE	5	31	20	0.84	0	0	46	10,219	27.4s
q2	Flat ReAct	1	5	4	0.50	0	0	0	473	6.1s
q2	Single LLM	1	1	0	0.00	0	0	0	6,338	3.9s
q3	SAGE	24	180	133	0.82	15	6	219	36,768	189.1s
q3	Flat ReAct	1	7	6	0.50	0	0	0	615	11.9s
q3	Single LLM	1	1	0	0.00	0	0	0	6,058	3.9s
q4	SAGE	6	35	22	0.84	0	0	64	12,869	30.7s
q4	Flat ReAct	1	5	4	0.50	0	0	0	723	5.5s
q4	Single LLM	1	1	0	0.00	0	0	0	7,778	4.5s
q5	SAGE	36	188	139	0.83	14	10	266	39,835	190.8s
q5	Flat ReAct	1	7	6	0.50	0	0	0	942	8.7s
q5	Single LLM	1	1	0	0.00	0	0	0	6,355	3.4s

Aggregate Summary by Condition

Table 4 aggregates the per-query results to compare the three conditions directly.

Table 4: Aggregate condition averages across all five queries. The gap between SAGE and the baselines is large and consistent across every metric.

Condition	Avg Conf	Avg Tools	Avg Cites	Avg Len	Avg Time	Total Retries	Total Re-plans
SAGE	0.834	67.6	131.6	22,237 chars	94.0s	30	16
Flat ReAct	0.500	5.0	0.0	673 chars	7.9s	0	0
Single LLM	0.000	0.0	0.0	6,496 chars	3.7s	0	0

Analysis

The ablation results reveal several consistent patterns and one important behavioral mode worth examining in detail.

Planning improves structure. SAGE decomposed queries into 5–6 sub-goals on q1, q2, and q4 with explicit dependencies, ensuring foundational concepts were resolved before comparative analysis. For q1, the dependency structure progressed from architectural differences (depth 0) through performance and computational trade-offs (depth 1) to empirical evidence (depth 2) and open research questions (depth 3). The Flat ReAct and Single LLM baselines produced monolithic outputs without this logical progression on every single query.

Tools improve grounding—decisively. Across all five SAGE runs, the agent made 20–139 tool calls per query, resulting in 46–266 cited URLs in the final reports (**658 citations in total**). In stark contrast, the Flat ReAct baseline produced **zero citations on every single query** despite having full tool access, and the Single LLM baseline (which has no tool access) likewise produced zero citations. This is the single largest gap in the study: SAGE produces substantively grounded output while baselines produce ungrounded prose, regardless of whether tools are nominally available. The planner's focused sub-questions are what make tools effective; without that structure, the ReAct loop's searches are too broad to yield usable evidence within its step budget.

The Critic enables self-correction. The Evidence Critic triggered retries on q1 (1 retry), q3 (15 retries), and q5 (14 retries), and triggered re-planning on q3 (6 re-plans) and q5 (10 re-plans). All other SAGE runs (q2, q4) passed every sub-goal on the first attempt with average confidence of 0.84. The variation across queries is informative: queries with focused, well-bounded scope (q2 hallucination, q4 quantum error correction) required no Critic intervention, while queries spanning multiple domains (q3 batteries: chemistry + environmental science + manufacturing; q5 RAG vs. fine-tuning: ML + deployment + economics) triggered extensive Critic feedback. Neither baseline has any self-correction capability: their confidence scores are constants (0.50 and 0.00 respectively).

Cost of planning. SAGE used 31–188 LLM calls and 27–191 seconds across queries, compared to 1 call and 3–4 seconds for Single LLM and 5–7 calls and 5–12 seconds for Flat ReAct. This is the expected trade-off: planning, multi-step reasoning, and quality critique require additional computation. The trade-off is substantial in absolute terms but justified by the resulting grounded output—no amount of additional compute on the baseline architectures produces citations.

Report quality scales with planning. SAGE's reports ranged from 10,219 to 39,835 characters with 46–266 citations, with an average of 22,237 characters per report. Single LLM reports averaged 6,496 characters with zero citations. Flat ReAct reports averaged just 673 characters—an order of magnitude shorter than even the Single LLM baseline—suggesting that without planning structure, the ReAct loop terminates prematurely with shallow answers when its tool calls don't surface useful evidence quickly.

Re-planning expansion (q3, q5). Two of five queries triggered substantial re-planning: q3 expanded to 24 nodes (6 re-plans, 15 retries) and q5 expanded to 36 nodes (10 re-plans, 14 retries). Notably, both expansion runs produced *the highest citation counts and longest reports in the entire study* (219 and 266 citations; 36,768 and 39,835 characters). This suggests that when re-planning triggers, it does succeed in gathering more evidence—but at a substantial computational cost (180–188 LLM calls vs. 31–37 for the well-behaved runs). We discuss this pattern in detail in Section 10.

Threats to Validity

Several factors limit the generalizability of these results. First, all five queries were authored by the team and may not represent the full range of research synthesis tasks. Second, the Flat ReAct baseline uses a fixed step count of 8, which may underperform relative to a tuned ReAct agent; however, this mirrors the design choice of allocating a comparable computational budget. Third, our metrics (confidence, citations, report length) are structural proxies for quality, not direct measures of factual accuracy or analytical depth—a proper evaluation would require expert human ratings. Fourth, all runs use a single LLM backend (LLaMA 3.3 70B via Groq); results may differ with other models. Fifth, each condition was run once per query in this sweep due to API cost and time constraints; variance across runs is not directly captured in this table.

Non-determinism evidence. A separate prior ablation run on the same queries produced markedly different behavior on two queries: q3 produced 0 citations (despite expanding to 36 nodes), while q5 ran cleanly with 6 nodes. In the present run, q3 produced 219 citations and q5 was the over-expansion case. The same code, same query, and same model produced very different DAG structures and citation counts across runs—a direct consequence of LLM stochasticity in the decomposition and Critic steps. This is a fundamental property of LLM-based agents and a meaningful threat to single-run evaluations of any agent system.

Complete Agent Trace

The following trace shows a representative SAGE execution for query q1 (*"Analyze the trade-offs between transformer-based and recurrent architectures for long-context reasoning, citing empirical evidence and identifying open research questions"*). This trace combines structural details from our q1 ablation run (5 nodes, 37 LLM calls, 24 tool calls, 1 retry, 32.0 seconds, 63 citations) with concrete tool sequences observed in our demo runs.

Full Agent Trace — q1

Phase 1: Query Decomposition

The planner decomposes the query into a 5-node DAG:

`sub_1`	Key architectural differences	depth 0	no deps
`sub_2`	Performance on long-context tasks	depth 1	depends on sub_1
`sub_3`	Computational efficiency trade-offs	depth 1	depends on sub_1
`sub_4`	Empirical evidence	depth 2	depends on sub_2, sub_3
`sub_5`	Open research questions	depth 3	depends on sub_2, sub_3, sub_4

Phase 2: Execution

Iteration 1 — sub_1 (depth 0, no dependencies):
Step 1: `web_search(``transformer vs RNN architecture'')` → 5 results
Step 2: `fetch_url(...)` → **failed** (HTTP 403 — paywall)
Step 3: `extract_claims(search results)` → 6 claims extracted
Steps 4–5: Additional web searches for depth
⇒ Max ReAct steps (5) reached; evidence accumulated from 3 successful tool calls
Critic: confidence = 0.80, contradictions = none, signal = `ACCEPT`

Iteration 2 — sub_3 (depth 1, sub_1 resolved):
Step 1: `web_search(``transformer RNN computational efficiency'')` → 5 results
Step 2: `extract_claims(...)` → 8 claims about memory/compute trade-offs
Step 3: `web_search(``RNN vs transformer memory requirements'')`
Step 4: `cross_reference(claims)` → all consistent, no contradictions
Step 5: `finish` — structured answer about parallelization advantages and memory costs
Critic: confidence = 0.65, signal = `RETRY` (1 retry triggered)

Iteration 3 — sub_3 retry (re-execution with revised query):
Steps 1–3: Refined search and claim extraction
Step 4: `finish` — expanded answer with quantitative complexity analysis
Critic: confidence = 0.85, signal = `ACCEPT`

Iteration 4 — sub_2 (depth 1, sub_1 resolved):
Steps 1–2: Two web searches on long-context reasoning benchmarks
Step 3: `finish` — answer citing transformer advantages in long-range dependency capture
Critic: confidence = 0.80, signal = `ACCEPT`

Iteration 5 — sub_4 (depth 2, sub_2 and sub_3 resolved):
Step 1: `web_search(``empirical comparison transformer RNN long context'')`
Step 2: `extract_claims(...)` → claims about recurrent memory augmentation studies
Step 3: `cross_reference(claims)` → consistent
Steps 4–5: Additional searches finding AAAI and arXiv papers
Critic: confidence = 0.85, signal = `ACCEPT`

Iteration 6 — sub_5 (depth 3, all predecessors resolved):
Context injected from sub_2, sub_3, sub_4 answers
Step 1: `web_search(``open research questions transformer long context'')`
Step 2: `extract_claims(...)` → claims about relational reasoning gaps
Step 3: `cross_reference(claims)` → consistent
Step 4: `finish` — answer identifying mechanisms, efficiency, and hybrid architecture questions
Critic: confidence = 0.82, signal = `ACCEPT`

All 5 nodes resolved.

Phase 3: Synthesis
Topological traversal order: sub_1 → sub_3 → sub_2 → sub_4 → sub_5
Findings collected in dependency order, provenance attached (63 source URLs tracked)
LLM synthesizes into coherent Markdown report with inline citations

Final Statistics: 5 nodes, 5 resolved, 37 LLM calls, 24 tool calls, 1 retry, 32.0 seconds, 63 citations

Failure Analysis and Limitations

We document several failure modes and limitations observed during development and evaluation. This section is critical for demonstrating genuine understanding of the system's boundaries and connecting to the course's themes about AI limitations.

Re-Planning Over-Expansion

The most significant behavioral pattern observed in our ablation is *re-planning over-expansion*, which manifested on two of five queries (q3 batteries; q5 RAG vs. fine-tuning). On these queries, the Critic repeatedly emitted `ESCALATE` signals, causing the re-planner to grow the DAG far beyond its initial decomposition: q3 expanded from ∼6 initial nodes to 24 nodes (6 re-plans, 15 retries), and q5 expanded to 36 nodes (10 re-plans, 14 retries). LLM call counts ballooned from ∼30 (typical) to 180 and 188 respectively.

Crucially, in this run *both expansion cases produced grounded output*: q3 yielded 219 citations in a 36,768-character report, and q5 yielded 266 citations in a 39,835-character report—the highest citation counts and longest reports in the entire study. This is in contrast to a prior ablation run on the same queries where q3 expanded to 36 nodes but produced 0 citations. The same code, same query, and same model produced fundamentally different outcomes across runs, depending on stochastic differences in LLM-generated decompositions, search results, and Critic confidence assessments.

The mechanism underlying expansion is a feedback loop between the Critic and the re-planner: the Critic rates evidence on cross-domain or breadth-heavy topics as insufficient, triggering `ESCALATE`; the re-planner inserts additional sub-goals to address gaps; those new sub-goals also receive borderline evidence on their own cross-domain questions; the cycle repeats. The current architecture has no hard cap on re-planning depth, no detector for diminishing returns, and no global budget on total node count. A simple fix—limiting re-plans to 3 per query and forcing synthesis with available evidence—would prevent unbounded expansion while preserving adaptive planning for moderate cases. This is a deliberate omission in the current implementation: we wanted to observe the unbounded behavior to characterize when and why it occurs, rather than masking it with conservative limits.

The non-determinism observation is itself an important finding. *The same query produced very different DAG structures across runs*: q3 expanded to 36 nodes with 0 citations on one run and 24 nodes with 219 citations on another. Single-run evaluations of LLM-based agents necessarily understate this variance. A more rigorous evaluation would require multiple seeds per query.

Other Observed Failure Modes

Tool Grounding Failures. The `fetch_url` tool fails on roughly 20% of URLs in our testing due to 403 errors, paywalls, or anti-bot protections. The agent adapts by falling back to additional web searches, but the depth of evidence is reduced. This is a concrete instance of tool grounding failure that no amount of LLM capability can resolve—the limitation is in the environment, not the model.

Decomposition Overlap. The LLM occasionally produces sub-questions that are semantically similar (e.g., "What are the computational requirements?" and "What are the efficiency trade-offs?"). The planner's validation catches malformed JSON and hallucinated dependencies, but it does not

currently detect semantic overlap. This contributes to redundant tool calls. Embedding-based deduplication (comparing sub-question embeddings and merging those above a similarity threshold) would address this.

Critic Calibration. The Evidence Critic's confidence scores are LLM-generated and not calibrated against ground truth. On well-behaved queries (q1, q2, q4), all sub-goals received confidence scores between 0.82 and 0.85, which may reflect the LLM's tendency toward optimistic self-assessment rather than genuine quality differences. On q3 and q5, the Critic was consistently more pessimistic, driving the over-expansion behavior. The deterministic overrides mitigate extreme miscalibration, but the fundamental limitation of LLM-as-judge remains.

Confirmation Bias. When LLM-generated search queries are informed by prior LLM-generated outputs, there is a risk of circular reinforcement. Our q1 run consistently surfaced evidence favoring transformers over RNNs, which may partly reflect the search query framing rather than the true balance of evidence. Adversarial query generation (explicitly searching for counterevidence) would address this bias.

Context Window Pressure. As the agent processes more sub-goals, accumulated history and context grow. On q5 with 36 nodes, this pressure is severe—the later nodes receive heavily truncated context, contributing to lower-quality evidence on the deepest nodes.

Structural Limitations

DAG Rigidity. The initial DAG structure, once constructed, can only be extended (via re-planning), not restructured. If the decomposition is fundamentally wrong, the agent cannot recover by reorganizing its sub-goals.

LLM Dependence. While the planning logic is external to the LLM, the system still depends on LLM quality for sub-question proposal, tool use reasoning, claim extraction, cross-referencing, and synthesis. A weak LLM would produce poor results regardless of the planning structure. This is the fundamental tension of neuro-symbolic systems: the symbolic structure provides guardrails, but the neural component provides the substance.

No Re-planning Depth Limit. As discussed above, the absence of a hard cap on re-planning iterations is the key vulnerability that allows over-expansion. A simple fix—limiting re-plans to 3 per query—would prevent the over-expansion failure mode while preserving the adaptive planning capability for moderate cases.

Evaluation Difficulty. Research synthesis quality is inherently subjective and difficult to evaluate automatically. Our metrics (confidence, citations, report length) are proxies for quality, not direct measures. A proper evaluation would require human expert ratings on dimensions like factual accuracy, coverage, coherence, and insight quality.

Connection to Course Themes

SAGE is designed as a living case study for the four modules of ESE 561:

Module 1 (Search & Planning). The DAG planner and priority scheduler directly instantiate classical planning concepts—state representation, action selection, and goal decomposition—in an LLM context. The scheduler's composite priority function combines depth-first traversal with

relevance scoring and confidence deficit, echoing informed search strategies from classical AI. The re-planner implements a form of plan repair [8] that is directly analogous to replanning in the planning literature. The stopping conditions (confidence threshold, max iterations, no eligible nodes) map directly to termination conditions in classical search algorithms. The q3 and q5 over-expansion behavior—where adaptive re-planning grew DAGs to 24 and 36 nodes respectively—mirrors the "search space explosion" problem that classical AI planners also face when search heuristics fail to converge.

Module 2 (Neural Models). The LLM serves as the reasoning and representation engine, but its outputs are always *interpreted and validated* by the agent layer. The ReAct loop demonstrates how neural generation can be grounded in external observations (tool results), transforming the LLM from a closed-book responder into an open-book researcher. The prompt templates in `sage/llm/prompts.py` show how LLM behavior can be shaped through careful interface design—the same model produces different behaviors (decomposition, reasoning, critique, synthesis) depending on the prompt structure.

Module 3 (Neuro-Symbolic Integration). SAGE exemplifies the neuro-symbolic paradigm: neural generation (LLM) + symbolic structure (DAG, rules, schemas). The Evidence Critic is the clearest example: the LLM provides a judgment (confidence score, contradiction list), but deterministic override rules enforce logical constraints (termination guarantees, consistency). The DAG itself is a symbolic structure that constrains and organizes neural reasoning—the LLM reasons freely within each node, but the planner determines which nodes exist and in what order they execute. The tool registry provides another symbolic interface: each tool has a fixed input/output schema that the LLM must conform to.

Module 4 (Limitations of AI). The failure analysis directly surfaces limitations of LLM-based systems. The 20% `fetch_url` failure rate is a concrete instance of tool grounding failure—the agent's reasoning is correct but the environment refuses access. The q3 and q5 re-planning over-expansion demonstrates that adaptive planning can fail to converge without proper bounds—a limitation shared by classical AI planners. The Critic's uncalibrated confidence scores illustrate the gap between LLM self-assessment and ground truth. The non-determinism observation (q3 went from 0 to 219 citations across runs of the same query) demonstrates that LLM-based agents are inherently stochastic and that single-run evaluation is fundamentally limited. These are not hypothetical limitations but observed behaviors from actual system runs.

Conclusion

SAGE demonstrates that explicit planning and critique layers, implemented as deterministic code external to the LLM, consistently improve the quality of automated research synthesis. Across all five test queries, SAGE produced higher-confidence, better-grounded, more structured outputs than both the Flat ReAct and Single LLM baselines, with 46–266 citations per query (658 total) compared to zero from any baseline run on any query. The Evidence Critic triggered self-correction (retries) on three of five queries and re-planning on two of five queries, directly demonstrating quality assurance mechanisms absent from both baselines.

Two queries (q3 and q5) triggered substantial re-planning expansion, growing their DAGs to 24 and 36 nodes respectively. Notably, both expansion cases produced the highest citation counts and longest reports in the entire study—suggesting that the re-planning mechanism, while computationally expensive when triggered, does succeed in gathering more evidence. A comparison to a prior run

on the same queries shows striking non-determinism: q3 produced 0 citations in one run and 219 in another. This finding is as valuable as the successes—it demonstrates that LLM-based agents are inherently stochastic and that bounded adaptive planning (with caps on re-planning depth and total node count) would be a meaningful improvement.

The ablation study confirms the value of each architectural layer: planning provides structure (5–36 focused sub-goals vs. monolithic output), tools provide grounding (46–266 citations vs. zero on every baseline run), and the Critic provides quality control (verified average confidence of 0.834 vs. unverified output). The Flat ReAct baseline's zero-citation result on every query, despite having full tool access, is particularly revealing: *tool access alone is not sufficient to produce grounded output*; the planner's focused sub-questions are what make tools effective.

The central lesson of this project connects directly to the course's guiding question: the intelligence of an LLM-based system comes from *both* the neural model and the surrounding structure, and the relative contribution of each can be measured through careful ablation. SAGE makes this boundary explicit and empirically interrogable.

Future Directions

Several extensions could strengthen the system: (1) a re-planning depth limit and global node budget to prevent unbounded over-expansion; (2) embedding-based semantic deduplication of sub-questions during decomposition; (3) adversarial query generation to reduce confirmation bias; (4) human-calibrated confidence scoring; (5) hierarchical summarization to manage context window pressure; (6) multi-seed evaluation to characterize the non-determinism observed across runs; and (7) support for multi-modal evidence in the synthesis pipeline.

References

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2210.03629

[2] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R.K.-W., and Lim, E.-P. "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models." *Association for Computational Linguistics (ACL)*, 2023. https://arxiv.org/abs/2305.04091

[3] Shinn, N., Cassano, F., Gopinath, A., Shukla, K., and Narasimhan, K. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2303.11366

[4] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2302.04761

[5] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2201.11903

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[7] Erol, K., Hendler, J., and Nau, D.S. "HTN Planning: Complexity and Expressivity." *AAAI Conference on Artificial Intelligence*, 1994. https://www.cs.umd.edu/~nau/papers/erol1994htn.pdf

[8] Fox, M., Gerevini, A., Long, D., and Serina, I. "Plan Stability: Replanning versus Plan Repair." *International Conference on Automated Planning and Scheduling (ICAPS)*, 2006. https://strathprints.strath.ac.uk/2776/